

Software Requirements Specification (SRS)

"Locate a Socket" – EV Charging Station Web Application

SIT725 | Task 2.1P | Version 1.0

By: MUHAMMAD NOUMAN QAISER

1. Introduction

1.1 Purpose

The functional and non-functional requirements are outlined in this Software Requirements Specification (SRS) document for the web application "Locate a Socket." Drivers of electric vehicles (EVs) can use the system to find local charging stations, access charging services, and make safe payments. Developers, designers, quality control staff, and project stakeholders involved in the system's development lifecycle are the target audience for this document.

1.2 Document Conventions

Functional requirements are prefixed FR and numbered hierarchically (e.g., FR1, FR1.1). Non-functional requirements are prefixed NFR (e.g., NFR1). Requirement priorities are labelled High (H), Medium (M), or Low (L). Figures are numbered sequentially.

1.3 Scope

Locate a Socket is a browser-based platform that allows EV drivers to search for available charging stations along a route, check real-time availability, obtain turn-by-turn navigation directions, and make secure online payments for charging services. Station owners can manage their listings, and administrators oversee platform operation and user management.

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
EV	Electric Vehicle
SRS	Software Requirements Specification
GPS	Global Positioning System
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
PCI-DSS	Payment Card Industry Data Security Standard
HTTPS	HyperText Transfer Protocol Secure
UI	User Interface

1.5 References

- IEEE. (2018). ISO/IEC/IEEE 29148:2018 – Systems and software engineering: Requirements engineering. IEEE Standards Association.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- OWASP Foundation. (2023). OWASP Top Ten. Retrieved from <https://owasp.org/www-project-top-ten/>
- PCI Security Standards Council. (2022). PCI DSS v4.0. Retrieved from <https://www.pcisecuritystandards.org/>

2. Overall Description

2.1 Product Perspective

Locate a Socket is a stand-alone web application that integrates with third-party payment gateways (Stripe or PayPal) and mapping APIs (Google Maps or Mapbox). It can be accessed without the need for extra software using contemporary web browsers on desktop and mobile devices. To deliver location-based services, the system uses either browser-based geolocation or the device's GPS.

2.2 Product Functions

- Search and display nearby EV charging stations on an interactive map.
- Show real-time station availability, pricing, and connector type (AC/DC).
- Provide turn-by-turn navigation (driving and walking modes) to selected stations.
- Allow users to start, monitor, and stop charging sessions.
- Process secure online payments and generate digital receipts.

- Support station management (owners) and system administration (admins).

2.3 User Classes and Characteristics

User Class	Description
EV Driver	Primary user. Searches stations, starts/stops charging sessions, and makes payments.
Station Owner	Registers, updates, and manages charging station listings and real-time availability.
System Administrator	Manages user accounts, monitors system logs, and resolves disputes.

2.4 Operating Environment

The application operates on modern web browsers (Chrome, Edge, Firefox, Safari) on desktop computers, laptops, smartphones, and tablets. A stable internet connection is required for real-time map data and payment processing. The backend is hosted on a cloud server and integrates with Mapping and Payment APIs via HTTPS.

2.5 Design and Implementation Constraints

- All client-server communication must use HTTPS.
- Payment processing must comply with PCI-DSS v4.0.
- The application must be mobile-responsive without requiring additional plugins.
- Backend must follow RESTful API architecture (Node.js / Express).
- Integration with third-party APIs introduces dependency and performance constraints.

2.6 Assumptions and Dependencies

- Users have internet-enabled devices with location services (GPS or browser geolocation) active.
- Station owners keep availability and pricing data up to date.
- Third-party Mapping and Payment Gateway services remain available and stable.
- Users possess valid payment methods (credit/debit card or digital wallet).

3. Specific Requirements

3.1 Functional Requirements

FR1 – User Registration and Authentication [H]

- FR1.1 Users shall be able to create an account using email and password.
- FR1.2 Users shall be able to log in securely; sessions managed via JWT tokens.
- FR1.3 Users shall be able to log out; the system shall support password reset via email.

FR2 – Location Search [H]

- FR2.1 The system shall detect the user's location using GPS or browser-based geolocation.
- FR2.2 Nearby charging stations shall be displayed within a user-configurable radius on the map.
- FR2.3 Users shall be able to search for stations by address or postcode.

FR3 – Station Information [H]

- FR3.1 The system shall display station name, address, connector type (AC Level 1/2 or DC Fast Charge), real-time availability status (available / occupied / offline), pricing (per kWh or per minute), and distance from the user.

FR4 – Navigation [H]

- FR4.1 Users shall be able to select a station from the map to view its detail page.
- FR4.2 The system shall provide turn-by-turn route directions (driving and walking) via a Mapping API.

FR5 – Charging Session Management [H]

- FR5.1 Users shall be able to start a charging session on an available point; the system shall track duration and energy consumed.
- FR5.2 Users shall be able to stop or terminate a charging session at any time.

FR6 – Payment System [H]

- FR6.1 Users shall be able to pay securely online (credit/debit card, digital wallet) at the end of a session.
- FR6.2 The system shall generate a downloadable receipt and prevent duplicate charges for the same session.

FR7 – Station Management (Station Owner) [M]

- FR7.1 Station owners shall be able to register a new station and update its details (availability, pricing, contact information).
- FR7.2 Station owners shall be able to deactivate or remove a station listing.

FR8 – Administration [M]

- FR8.1 Administrators shall be able to manage user accounts (create, suspend, delete).
- FR8.2 Administrators shall be able to view system logs, usage statistics, and resolve reported disputes.

3.2 External Interface Requirements

User Interface

- Map-based view for searching and selecting charging stations.
- Search and filter panel (by connector type, availability, price range, distance).
- Station detail page with real-time status and navigation button.
- Payment screen, confirmation, and downloadable receipt.
- Account management pages for all user classes.

Software Interfaces

- Mapping API: Google Maps Platform or Mapbox (station display, routing).
- Payment Gateway: Stripe or PayPal (secure transaction processing).
- Database: PostgreSQL (users, stations, sessions, transactions, logs).
- Backend: Node.js / Express RESTful API; JSON data exchange; JWT authentication.

Communication Interfaces

- All communication over HTTPS (TLS 1.2 or higher).
- JSON-format data exchange between frontend and backend.
- Secure token-based (JWT) authentication for all protected API endpoints.

3.3 Non-Functional Requirements

ID	Requirement	Acceptance Criterion
NFR1	Performance – system response time	≤ 3 seconds under normal load
NFR2	Security – authentication and sessions	OWASP Top 10 compliant
NFR3	Payment security – encrypted transactions	PCI-DSS v4.0 compliant

NFR4	Usability – task completion in ≤ 3 steps	Mobile and desktop responsive
NFR5	Reliability – system availability	99% uptime target
NFR6	Scalability – horizontal scaling support	Containerised, load-balanced

4. Supporting Models

4.1 Use Case Diagram

[Figure 1](#) illustrates the interactions between the three system actors EV Driver, Station Owner, and Administrator and the primary use cases of the Locate a Socket application. The EV Driver engages with the majority of system functionality, from registration through to payment. Station Owners manage their listings, while the Administrator oversees user accounts and system health.

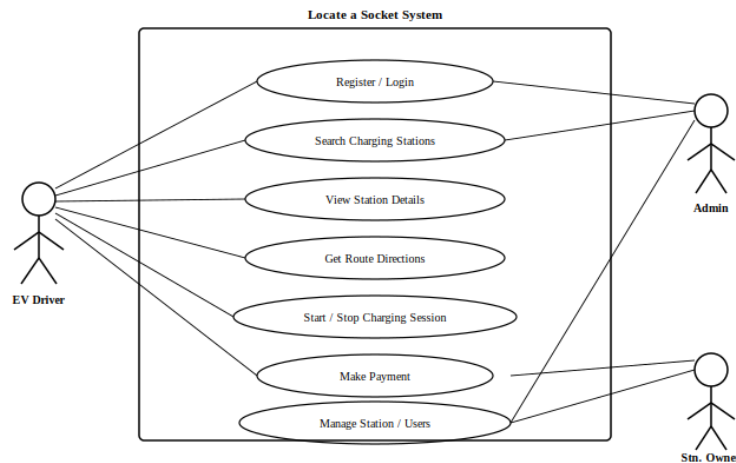
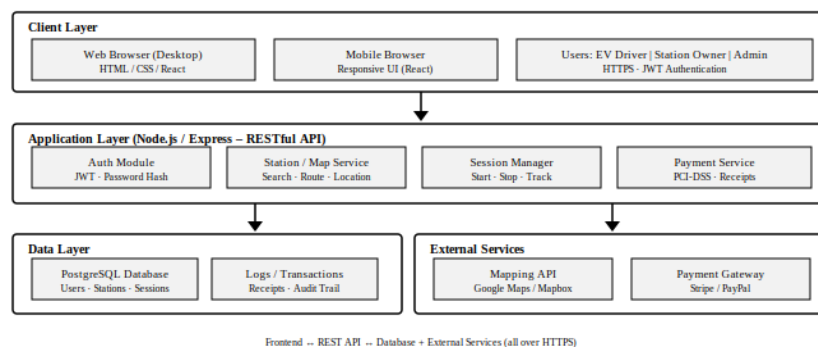


Figure 1 – Use Case Diagram: Locate a Socket

4.2 System Architecture Diagram

[Figure 2](#) shows the three-tier architecture. The Client Layer (responsive browser UI built with React) communicates over HTTPS with the Application Layer (Node.js / Express RESTful API), which handles business logic, authentication, session management, and payment processing. The Data Layer stores persistent data in a PostgreSQL database. External Mapping and Payment Gateway APIs are called by the backend over secure HTTPS connections.



Frontend ↔ REST API ↔ Database ↔ External Services (all over HTTPS)

Figure 2 – System Architecture Diagram: Locate a Socket

4.3 References

- IEEE. (2018). ISO/IEC/IEEE 29148:2018 – Systems and software engineering: Life cycle processes – Requirements engineering. IEEE Standards Association.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- OWASP Foundation. (2023). OWASP Top Ten. Retrieved from <https://owasp.org/www-project-top-ten/>
- PCI Security Standards Council. (2022). PCI DSS v4.0. Retrieved from <https://www.pcisecuritystandards.org/>
- Google. (2024). Google Maps Platform Documentation. Retrieved from <https://developers.google.com/maps>
- Stripe, Inc. (2024). Stripe Payments API Documentation. Retrieved from <https://stripe.com/docs>